



# **ANKARA ÜNİVERSİTESİ MÜHENDİSLİK FAKÜLTESİ BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ**

**BLM4522 Ağ Tabanlı Paralel Dağıtım Sistemleri Final Raporu**

**Proje 1: Veritabanı Performans Optimizasyonu ve İzleme**

**Proje 3: Veritabanı Güvenliği ve Erişim Kontrolü**

**Proje 6: Veritabanı Yükseltme ve Sürüm Yönetimi**

*Kullanılan Veritabanı: PostgreSQL 16 – demo (PostgresPro Havayolu Rezervasyon Veritabanı, bookings şeması)  
(<https://postgrespro.com/community/demodb>)*

**Bekir Top**

**22290383**

**2025–2026 Bahar Dönemi**

<https://github.com/bekirtop/BLM4522>

**Videolar;**

Proje 1 : <https://drive.google.com/file/d/1anICvUtOo03SDkeQPZCZqRiTStWPQm25/view?usp=sharing>

Proje 3 : [https://drive.google.com/file/d/1SuGLAXXUTavQLqmvXqd1WMFZKqlpuo\\_s/view?usp=sharing](https://drive.google.com/file/d/1SuGLAXXUTavQLqmvXqd1WMFZKqlpuo_s/view?usp=sharing)

Proje 6 : <https://drive.google.com/file/d/1qhSdWqXCz2nbUzr5xaz1K53tY5aNWkU4/view?usp=sharing>

# İÇİNDEKİLER

## **PROJE 1: VERİTABANI PERFORMANS OPTİMİZASYONU VE İZLEME**

1. Giriş
2. Çalışma Ortamı ve Kullanılan Veritabanı
3. Veritabanı İzleme – pg\_stat\_statements
4. İndeks Yönetimi
5. Sorgu İyileştirme
6. Rol ve Erişim Yönetimi
7. Sonuç ve Değerlendirme

## **PROJE 3: VERİTABANI GÜVENLİĞİ VE ERİŞİM KONTROLÜ**

1. Giriş
2. Çalışma Ortamı ve Kullanılan Veritabanı
3. Erişim Yönetimi – Roller ve Yetkilendirme
4. Veri Şifreleme – pgcrypto
5. SQL Injection Testleri ve Korunma Yöntemleri
6. Audit Log – Denetim Kayıtları
7. Sonuç ve Değerlendirme

## **PROJE 6: VERİTABANI YÜKSELTME VE SÜRÜM YÖNETİMİ**

1. Giriş
2. Çalışma Ortamı ve Kullanılan Veritabanı
3. Sürüm Yönetimi – Şema Versiyon Takibi ve DDL Event Trigger
4. Yükseltme Planı – v1.0.0 → v2.0.0
5. Test ve Geri Dönüş Planı
6. Sonuç ve Değerlendirme

# PROJE 1: VERİTABANI PERFORMANS OPTİMİZASYONU VE İZLEME

## 1. GİRİŞ

Bu projede, PostgreSQL 16 üzerinde çalışan demo veritabanının (PostgresPro örnek havayolu rezervasyon veritabanı, bookings şeması) performans izleme, indeksleme ve sorgu iyileştirme süreçleri MS SQL Server'daki karşılık gelen kavramlarla (SQL Profiler, Execution Plan, Database Engine Tuning Advisor, sp\_who2 vb.) ilişkilendirilerek ele alınmıştır. Çalışmanın amacı; veritabanı seviyesinde performans darboğazlarının PostgreSQL'in yerleşik istatistik görünümüleri ve EXPLAIN ANALYZE komutu ile tespit edilmesi, uygun indekslerin tasarlanarak uygulanması, sorgu yazım alışkanlıklarının iyileştirilmesi ve son olarak rol bazlı erişim yönetiminin kurulmasıdır.

Proje dört adımdan oluşmaktadır: (1) veritabanı izleme, (2) indeks yönetimi, (3) sorgu iyileştirme ve (4) rol ve erişim yönetimi. Her adımda önce mevcut durum EXPLAIN ANALYZE ile ölçülmüş, ardından iyileştirme uygulanmış ve sonuç tekrar ölçülerek öncesi/sonrası karşılaştırması yapılmıştır.

## 2. ÇALIŞMA ORTAMI VE KULLANILAN VERİTABANI

Çalışma ortamı PostgreSQL 16 ve psql komut satırı istemcisinden oluşmaktadır. Performans ve indeks etkisinin EXPLAIN ANALYZE çıktılarında net biçimde gözlemlenebilmesi için PostgresPro'nun "demo-medium" ölçeğindeki örnek havayolu rezervasyon veritabanı kullanılmıştır. Bu veritabanındaki tablo boyutları yaklaşık olarak şu şekildedir:

- bookings.ticket\_flights → 1.045.726 satır
- bookings.seats → 1.339.832 satır
- bookings.tickets → 366.733 satır
- bookings.bookings → 262.788 satır
- bookings.flights → 33.121 satır
- bookings.airports\_data → 104 satır
- bookings.aircrafts\_data → 9 satır

Bu ölçekteki tablolar üzerinde PostgreSQL'in sorgu planlayıcısı (query planner), küçük tablolarda olduğu gibi her zaman tam tablo taraması (Seq Scan) tercih etmek yerine, indeks mevcut olduğunda Index Scan / Bitmap Heap Scan gibi daha hızlı erişim yöntemlerine geçebilmektedir. Bu nedenle indeks etkisi gerçekçi biçimde ölçülebilmektedir.

## 3. VERİTABANI İZLEME – pg\_stat\_statements

PostgreSQL'de MS SQL Server'daki SQL Profiler / Query Store aracının karşılığı pg\_stat\_statements eklentisidir. Bu eklenti, sunucuda çalıştırılan her sorgunun çağrı sayısını, ortalama ve toplam çalışma sürelerini tutar. 01\_veritabanı\_izleme.sql dosyasında önce eklenti etkinleştirilmiş, ardından en yavaş sorgular, tablo erişim istatistikleri, aktif bağlantılar ve veritabanı önbellek (cache) isabet oranı sorgulanmıştır.

```
-- 1. pg_stat_statements eklentisini aktif et
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

-- 2. En yavaş 10 sorguyu listele (ortalama süreye göre)
SELECT
  LEFT(query, 80)          AS sorgu,
  calls                    AS cagri_sayisi,
  ROUND(mean_exec_time::numeric, 3) AS ort_sure_ms,
  ROUND(total_exec_time::numeric, 3) AS toplam_sure_ms
FROM pg_stat_statements
ORDER BY mean_exec_time DESC
LIMIT 10;
```

Konsol çıktısı:

```
demo=# - psql (PostgreSQL 16)
sorgu | cagri_sayisi | ort_sure_ms | toplam_sure_ms
-----|-----|-----|-----
SELECT f.departure_airport AS kalkis_havalimani, COUNT(DISTINCT | 3 | 842.156 | 2526.468
SELECT b.book_ref, b.total amount, (SELECT COUNT(*) FROM booking | 2 | 615.903 | 1231.806
SELECT * FROM bookings.ticket_flights | 4 | 198.567 | 794.268
SELECT tf.ticket_no, tf.flight_id, tf.amount FROM bookings.ticke | 2 | 161.397 | 322.794
SELECT flight_id, flight_no, scheduled_departure FROM bookings.f | 2 | 45.218 | 90.436
(10 rows)
```

Tablo erişim istatistikleri (pg\_stat\_user\_tables) sorgusu, her tablonun kaç kez tam tablo taraması (seq\_scan) ve kaç kez indeks üzerinden erişim (idx\_scan) ile okunduğunu gösterir. MS SQL Server'daki sys.dm\_db\_index\_usage\_stats görünümüne karşılık gelir:

```
SELECT
    schemaname AS sema, relname AS tablo_adi,
    seq_scan AS tam_tablo_tarama, idx_scan AS index_ile_erisim,
    n_live_tup AS satir_sayisi
FROM pg_stat_user_tables
ORDER BY seq_scan DESC LIMIT 10;
```

| sema     | tablo_adi      | tam_tablo_tarama | index_ile_erisim | satir_sayisi |
|----------|----------------|------------------|------------------|--------------|
| bookings | seats          | 12               | 0                | 1339832      |
| bookings | ticket_flights | 9                | 4                | 1045726      |
| bookings | tickets        | 6                | 3                | 366733       |
| bookings | bookings       | 5                | 2                | 262788       |
| bookings | flights        | 7                | 5                | 33121        |
| bookings | airports_data  | 4                | 0                | 104          |
| bookings | aircrafts_data | 3                | 0                | 9            |

Son olarak veritabanı genel istatistikleri sorgulanarak önbellek isabet oranı (cache hit ratio) hesaplanmıştır. %99'un üzerindeki bir oran, sık erişilen verinin diskten değil bellekten okunduğunu, dolayısıyla I/O yükünün düşük olduğunu gösterir:

| veritabanı | onbellekten_okunan | diskten_okunan | cache_hit_orani_yuzde | döndürülen_satir | getirilen_satir |
|------------|--------------------|----------------|-----------------------|------------------|-----------------|
| demo       | 8423156            | 21304          | 99.75                 | 5482190          | 3127845         |

## 4. İNDEKS YÖNETİMİ

İndeks, bir kitaptaki dizine benzer biçimde aranan değerin tam konumunu gösterir ve tüm tablonun taranmasını gereksiz kılar. Bu adımda önce mevcut indeksler listelenmiş, ardından indeks olmadan ve indeks eklendikten sonra aynı sorgular EXPLAIN ANALYZE ile çalıştırılarak performans farkı ölçülmüştür.

### 4.1. Tek Kolonlu İndeks - ticket\_flights.amount

ticket\_flights tablosunda amount > 50000 koşuluyla yapılan filtreleme, indeks yokken tüm 1.045.726 satırın taranmasını gerektirmektedir:

```
EXPLAIN ANALYZE
SELECT tf.ticket_no, tf.flight_id, tf.amount
FROM bookings.ticket_flights tf
WHERE tf.amount > 50000;
```

```
demo=# - psql (PostgreSQL 16)
QUERY PLAN
-----
Seq Scan on ticket_flights tf (cost=0.00..27726.08 rows=94115 width=21)
    (actual time=0.020..156.842 rows=93824 loops=1)
    Filter: (amount > '50000'::numeric)
    Rows Removed by Filter: 951902
Planning Time: 0.112 ms
Execution Time: 161.397 ms
(6 rows)
```

amount kolonu üzerinde bir indeks oluşturulduktan sonra aynı sorgu tekrar çalıştırılmıştır:

```
CREATE INDEX idx_ticket_flights_amount
ON bookings.ticket_flights (amount);
```

```
demo=# - psql (PostgreSQL 16)
QUERY PLAN
-----
Bitmap Heap Scan on ticket_flights tf (cost=1142.23..14523.67 rows=94115 width=21)
    (actual time=8.214..27.563 rows=93824 loops=1)
    Recheck Cond: (amount > '50000'::numeric)
    Heap Blocks: exact=8420
    -> Bitmap Index Scan on idx_ticket_flights_amount (cost=0.00..1118.70 rows=94115 width=0)
        (actual time=6.452..6.452 rows=93824 loops=1)
        Index Cond: (amount > '50000'::numeric)
Planning Time: 0.098 ms
Execution Time: 31.205 ms
(8 rows)
```

Sonuç: çalışma süresi 161,4 ms'den 31,2 ms'ye düşmüş, yani yaklaşık 5 kat hızlanma elde edilmiştir. Sorgu planlayıcısı, indeks sayesinde 1 milyondan fazla satırı tek tek taramak yerine yalnızca koşulu sağlayan ~94.000 satıra doğrudan erişmiştir.

## 4.2. Tarih Aralığı İndeksi – flights.scheduled\_departure

flights tablosunda belirli bir tarih aralığındaki uçuşları filtreleyen sorgu için de aynı yöntem uygulanmıştır:

```
-- İndeks öncesi
EXPLAIN ANALYZE
SELECT flight_id, flight_no, scheduled_departure, status
FROM bookings.flights
WHERE scheduled_departure BETWEEN '2017-01-01' AND '2017-06-01';

-- İndeks ekle
CREATE INDEX idx_flights_departure
ON bookings.flights (scheduled_departure);
```

```
demo=# - psql (PostgreSQL 16)
İndeks ÖNCESİ:
Seq Scan on flights (cost=0.00..1234.51 rows=14237 width=23)
    (actual time=0.015..3.842 rows=14296 loops=1)
    Filter: (scheduled_departure BETWEEN '2017-01-01' AND '2017-06-01')
    Rows Removed by Filter: 18825
    Execution Time: 4.215 ms

İndeks SONRASI:
Index Scan using idx_flights_departure on flights (cost=0.42..612.34 rows=14237 width=23)
    (actual time=0.025..1.124 rows=14296 loops=1)
    Index Cond: (scheduled_departure BETWEEN '2017-01-01' AND '2017-06-01')
    Execution Time: 1.389 ms
```

## 4.3. Bileşik (Composite) İndeks

Hem status hem de scheduled\_departure kolonlarına göre filtreleme yapan sorgular için iki kolonlu bir bileşik indeks oluşturulmuştur:

```
CREATE INDEX idx_flights_status_departure
ON bookings.flights (status, scheduled_departure);

EXPLAIN ANALYZE
SELECT flight_id, flight_no, scheduled_departure
FROM bookings.flights
WHERE status = 'Arrived'
AND scheduled_departure > '2017-06-01';
```

```
demo=# - psql (PostgreSQL 16)
QUERY PLAN
-----
Index Scan using idx_flights_status_departure on flights (cost=0.42..145.21 rows=3142 width=23)
    (actual time=0.022..0.487 rows=3158 loops=1)
    Index Cond: ((status = 'Arrived') AND (scheduled_departure > '2017-06-01'))
    Planning Time: 0.103 ms
    Execution Time: 0.612 ms
(4 rows)
```

## 4.4. Kullanılmayan İndekslerin Tespiti ve Boyut Karşılaştırması

idx\_scan değeri 0 olan indeksler hiç kullanılmamış demektir; bu indeksler disk alanı kaplamakla kalmaz, aynı zamanda her INSERT/UPDATE/DELETE işleminde güncellenmeleri gerektiğinden yazma performansını da düşürürler. Bu sorgu çalıştırıldığında, projede oluşturulan tüm indekslerin en az bir kez kullanılmış olması nedeniyle sonuç kümesi boş dönmüştür — bu da oluşturulan indekslerin gerçekten ihtiyaca yönelik olduğunu doğrulamaktadır:

```
SELECT schemaname AS sema, relname AS tablo, indexrelname AS index_adi,
       idx_scan AS kullanim_sayisi,
       pg_size_pretty(pg_relation_size(indexrelid)) AS boyut
FROM pg_stat_user_indexes
WHERE idx_scan = 0
ORDER BY pg_relation_size(indexrelid) DESC;
```

```
demo=# - psql (PostgreSQL 16)
sema | tablo | index_adi | kullanim_sayisi | boyut
-----+-----+-----+-----+-----
(0 rows)
```

Tablo ve indeks boyutları karşılaştırıldığında, en büyük indeks yüküne sahip tabloların ticket\_flights ve seats olduğu görülmektedir; bu da en yoğun erişilen tablolar üzerinde indeks bakımının özellikle önemli olduğunu göstermektedir:

| nesne          | toplam boyut | tablo boyutu | index boyutu |
|----------------|--------------|--------------|--------------|
| ticket_flights | 104 MB       | 89 MB        | 15 MB        |
| seats          | 98 MB        | 72 MB        | 26 MB        |
| tickets        | 62 MB        | 45 MB        | 17 MB        |
| bookings       | 38 MB        | 28 MB        | 10 MB        |
| flights        | 6960 kB      | 4416 kB      | 2544 kB      |
| airports_data  | 104 kB       | 40 kB        | 64 kB        |
| aircrafts_data | 48 kB        | 16 kB        | 32 kB        |

(7 rows)

## 5. SORGU İYİLEŞTİRME

Bu adımda sık karşılaşılan altı kötü sorgu kalıbı ele alınmış; her biri için önce sorunlu (kötü) versiyon EXPLAIN ANALYZE ile çalıştırılmış, ardından iyileştirilmiş (iyi) versiyonu ile karşılaştırılmıştır.

### 5.1. SELECT \* Kullanımının Önlenmesi

SELECT \* tüm kolonları çektiği için ağ trafiğini ve bellek kullanımını artırır. Sadece gerekli kolonların seçilmesi performansı doğrudan etkiler:

```
-- KÖTÜ: EXPLAIN ANALYZE SELECT * FROM bookings.ticket_flights;
-- İYİ : EXPLAIN ANALYZE SELECT ticket_no, flight_id, amount
--       FROM bookings.ticket_flights;
```

```
demo=# - psql (PostgreSQL 16)
KÖTÜ -> Execution Time: 287.412 ms (width=33, tüm kolonlar diskten okunur)
İYİ -> Execution Time: 198.567 ms (width=21, yalnızca 3 kolon okunur)
```

### 5.2. WHERE İçinde Fonksiyon Kullanımının Önlenmesi

WHERE koşulunda bir kolon üzerinde fonksiyon çağrılırsa (örn. EXTRACT), o kolon üzerindeki standart bir indeks kullanılamaz ve PostgreSQL Seq Scan'e döner. Aralık (range) tabanlı bir filtreye dönüştürmek indeksin tekrar devreye girmesini sağlar:

```
-- KÖTÜ: indeks devre dışı kalır
WHERE EXTRACT(MONTH FROM scheduled_departure) = 6

-- İYİ: idx_flights_departure indeksini kullanabilir
WHERE scheduled_departure >= '2017-06-01'
AND scheduled_departure < '2017-07-01'
```

```
demo=# - psql (PostgreSQL 16)
KÖTÜ -> Seq Scan on flights Execution Time: 4.873 ms (Rows Removed by Filter: 30547)
İYİ -> Index Scan using idx_flights_departure Execution Time: 0.498 ms
```

### 5.3. Subquery Yerine JOIN Kullanımı

Her satır için ayrı bir alt sorgu (correlated subquery) çalıştırmak yerine, JOIN ve GROUP BY ile tek geçişte sonuç üretmek çok daha verimlidir:

```
-- KÖTÜ: Her bookings satırı için ayrı COUNT alt sorgusu
SELECT b.book_ref, b.total_amount,
       (SELECT COUNT(*) FROM bookings.tickets t
        WHERE t.book_ref = b.book_ref) AS bilet_sayisi
FROM bookings.bookings b
LIMIT 1000;

-- İYİ: Tek geçişte LEFT JOIN + GROUP BY
SELECT b.book_ref, b.total_amount, COUNT(t.ticket_no) AS bilet_sayisi
FROM bookings.bookings b
LEFT JOIN bookings.tickets t ON b.book_ref = t.book_ref
GROUP BY b.book_ref, b.total_amount
LIMIT 1000;
```

```
demo=# - psql (PostgreSQL 16)
KÖTÜ -> Execution Time: 615.903 ms (1000 adet alt sorgu = 1000 ekstra tarama)
İYİ -> Execution Time: 47.281 ms (tek HashAggregate planı)
```

### 5.4. LIKE İfadesinde Joker Karakter Konumu

LIKE ifadesinde değer % ile başlıyorsa (örn. '%IVAN%'), B-Tree indeksi kullanılamaz ve tüm tablo taranır. Aranılan değer sabit bir önek ile başlıyorsa (örn. 'IVAN%') indeks yeniden devreye girebilir:

```
-- KÖTÜ: baştaki % nedeniyle indeks kullanılamaz
EXPLAIN ANALYZE
SELECT passenger_id, passenger_name
FROM bookings.tickets
WHERE passenger_name LIKE '%IVAN%';

-- İYİ: sabit örnek ile başlıyor
EXPLAIN ANALYZE
SELECT passenger_id, passenger_name
FROM bookings.tickets
WHERE passenger_name LIKE 'IVAN%';
```

```
demo=# - psql (PostgreSQL 16)
KÖTÜ -> Seq Scan on tickets (actual rows=1822 loops=1)
      Execution Time: 58.214 ms
İYİ -> Seq Scan on tickets (actual rows=311 loops=1)
      Execution Time: 21.045 ms

Not: passenger_name üzerinde indeks bulunmadığı için her iki durumda da Seq Scan
görülür; ancak 'IVAN%' örneğinin daha az satırla eşleşmesi taranan satır sayısını ve
süreyi azaltır. Sık kullanılan örnek aramalarında text pattern_ops indeksi önerilir.
```

## 5.5. DISTINCT Yerine EXISTS Kullanımı

DISTINCT, sonuç kümesinin tamamını bellekte tekilleştirdiği için maliyetlidir. Sadece bir koşulu sağlayan kayıtların var olup olmadığını kontrol etmek için EXISTS, ilk eşleşmede durduğundan daha verimlidir:

```
-- KÖTÜ
SELECT DISTINCT flight_id FROM bookings.ticket_flights WHERE amount > 10000;

-- İYİ
SELECT flight_id FROM bookings.flights f
WHERE EXISTS (
  SELECT 1 FROM bookings.ticket_flights tf
  WHERE tf.flight_id = f.flight_id AND tf.amount > 10000
);
```

```
demo=# - psql (PostgreSQL 16)
KÖTÜ -> HashAggregate (DISTINCT) Execution Time: 312.654 ms
İYİ -> Seq Scan + EXISTS (Subplan kısa devre) Execution Time: 84.327 ms
```

## 5.6. Gerçekçi Rapor Sorgusu ve VACUUM ANALYZE

Son olarak; departure\_airport, ticket\_flights ve flights üzerinde JOIN + GROUP BY ile havalimanı bazında gelir raporu üretilmiş, sorgunun ihtiyaç duyduğu indeksler önceden oluşturulmuştur:

```
CREATE INDEX IF NOT EXISTS idx_flights_departure_airport
ON bookings.flights (departure_airport);
CREATE INDEX IF NOT EXISTS idx_ticket_flights_flight_id
ON bookings.ticket_flights (flight_id);

SELECT f.departure_airport AS kalkis_havalimani,
       COUNT(DISTINCT f.flight_id) AS ucus_sayisi,
       COUNT(tf.ticket_no) AS bilet_sayisi,
       ROUND(SUM(tf.amount)) AS toplam_gelir,
       ROUND(AVG(tf.amount)) AS ort_bilet_ucreti
FROM bookings.flights f
JOIN bookings.ticket_flights tf ON f.flight_id = tf.flight_id
WHERE f.status = 'Arrived'
GROUP BY f.departure_airport
ORDER BY toplam_gelir DESC
LIMIT 20;
```

```
demo=# - psql (PostgreSQL 16)
kalkis_havalimani | ucus_sayisi | bilet_sayisi | toplam_gelir | ort_bilet_ucreti
-----+-----+-----+-----+-----
SVO                |         612 |        19840 |    412589300 |          20796
DME                |         498 |        16210 |    337104500 |          20796
LED                |         355 |        11532 |    238765200 |          20704
OVB                |         203 |         6587 |    136104900 |          20662
(...)             |         ... |         ... |           ... |           ...
(20 rows)
```

Sorgu çalıştırıldıktan sonra istatistiklerin güncel kalması için ilgili tablolarda VACUUM ANALYZE çalıştırılmıştır; bu işlem sorgu planlayıcısının gerçek veri dağılımına göre doğru kararlar almasını sağlar:

```
VACUUM ANALYZE bookings.ticket_flights;
VACUUM ANALYZE bookings.flights;
VACUUM ANALYZE bookings.bookings;
```

## 6. ROL VE ERİŞİM YÖNETİMİ

Veritabanı güvenliğinin temeli, kullanıcılara işleri için gerekli olan en az yetkinin (least privilege) verilmesidir. Bu adımda üç farklı kullanıcı profili tanımlanmıştır: yalnızca rapor okuyan analist, veri girişi yapan operatör ve tüm yetkilere sahip db\_yoneticisi.

```
-- Sadece okuma yetkili analist
CREATE ROLE analist LOGIN PASSWORD 'Analist2024!';
GRANT CONNECT ON DATABASE demo TO analist;
GRANT USAGE ON SCHEMA bookings TO analist;
GRANT SELECT ON ALL TABLES IN SCHEMA bookings TO analist;

-- Veri girişi/güncelleme yapabilen operatör
CREATE ROLE operatör LOGIN PASSWORD 'Operator2024!';
GRANT SELECT, INSERT, UPDATE ON bookings.bookings TO operatör;
GRANT SELECT, INSERT, UPDATE ON bookings.tickets TO operatör;
GRANT SELECT, INSERT, UPDATE ON bookings.ticket_flights TO operatör;
GRANT SELECT ON bookings.flights TO operatör;

-- Tüm yetkilere sahip yönetici
CREATE ROLE db_yoneticisi LOGIN PASSWORD 'Admin2024!';
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA bookings TO db_yoneticisi;
```

Yetkiler information\_schema.role\_table\_grants üzerinden doğrulanmıştır:

| kullanici     | sema     | tablo          | yetki  |
|---------------|----------|----------------|--------|
| analist       | bookings | aircrafts_data | SELECT |
| analist       | bookings | airports_data  | SELECT |
| analist       | bookings | bookings       | SELECT |
| analist       | bookings | flights        | SELECT |
| ...           | ...      | ...            | ...    |
| operatör      | bookings | bookings       | INSERT |
| operatör      | bookings | bookings       | SELECT |
| operatör      | bookings | bookings       | UPDATE |
| operatör      | bookings | flights        | SELECT |
| db_yoneticisi | bookings | bookings       | DELETE |
| db_yoneticisi | bookings | bookings       | INSERT |
| db_yoneticisi | bookings | bookings       | SELECT |
| db_yoneticisi | bookings | bookings       | UPDATE |
| (...)         |          |                |        |

analist rolüyle bağlanıldığında SELECT işlemleri normal şekilde çalışırken, DELETE gibi yetkisi olmayan bir komut çalıştırılmaya çalışıldığında PostgreSQL beklenen şekilde "permission denied" hatası döndürmüştür:

```
demo=# - psql (PostgreSQL 16)
demo=> SET search_path TO bookings;
demo=> SELECT COUNT(*) FROM bookings;
count
-----
262788
(1 row)

demo=> DELETE FROM bookings WHERE 1=1;
ERROR: permission denied for table bookings
```

Ek bir güvenlik katmanı olarak flights tablosunda Row Level Security (RLS) etkinleştirilmiş; db\_yoneticisi tüm satırları görebilirken, analist yalnızca status = 'Arrived' olan uçuşları görebilecek şekilde politika tanımlanmıştır:

```
ALTER TABLE bookings.flights ENABLE ROW LEVEL SECURITY;

CREATE POLICY yönetici_politikasi ON bookings.flights
FOR ALL TO db_yoneticisi USING (true);

CREATE POLICY analist_politikasi ON bookings.flights
FOR SELECT TO analist USING (status = 'Arrived');
```

Son olarak aşırı bağlantı kullanımını önlemek amacıyla her role bağlantı limiti tanımlanmıştır (analist: 5, operatör: 10, db\_yoneticisi: sınırsız):

| rolname       | baglanti_limiti |
|---------------|-----------------|
| analist       | 5               |
| db_yoneticisi | -1              |
| operatör      | 10              |
| (3 rows)      |                 |

## 7. SONUÇ VE DEĞERLENDİRME

Bu projede PostgreSQL 16 üzerinde çalışan demo (bookings) veritabanı için uçtan uca bir performans optimizasyonu ve izleme süreci uygulanmıştır. pg\_stat\_statements ve pg\_stat\_user\_tables görünümleri ile darboğaz oluşturan sorgular ve tablolar tespit edilmiş; ticket\_flights.amount, flights.scheduled\_departure ve (status, scheduled\_departure) bileşik kolonları üzerine eklenen indekslerle sorgu sürelerinde 5 kata varan iyileşme sağlanmıştır. Sorgu iyileştirme bölümünde altı yaygın anti-pattern (SELECT \*, fonksiyonlu WHERE, subquery, baştan joker LIKE, gereksiz DISTINCT) ele alınarak her biri için daha verimli alternatif sunulmuştur. Son olarak analist, operatör ve db\_yoneticisi rolleriyle en az yetki ilkesine dayalı bir erişim modeli ve Row Level Security politikaları kurularak, performans çalışmalarının güvenlik gereksinimleriyle birlikte ele alınması gerektiği gösterilmiştir.

Gelecek çalışmalarda aşağıdaki geliştirmeler yapılabilir:

- pg\_stat\_statements verileri periyodik olarak bir izleme tablosuna kaydedilerek zaman içindeki performans trendleri analiz edilebilir.
- Kısmi (partial) ve ifade (expression) indeksleri ile daha özel sorgu kalıpları hızlandırılabilir.
- pg\_cron veya benzeri bir zamanlayıcı ile VACUUM ANALYZE işlemleri otomatikleştirilebilir.
- RLS politikaları, gerçek bir uygulama katmanındaki oturum (session) bilgisiyle entegre edilebilir.

# PROJE 3: VERİTABANI GÜVENLİĞİ VE ERİŞİM KONTROLÜ

## 1. GİRİŞ

Bu projede, PostgreSQL 16 üzerinde çalışan demo veritabanı (bookings şeması) üzerinde uçtan uca bir veritabanı güvenliği ve erişim kontrolü senaryosu uygulanmıştır. Çalışma dört bölümden oluşmaktadır: (1) rol bazlı erişim yönetimi ve en az yetki ilkesi, (2) pgcrypto eklentisi ile hassas verilerin şifrlenmesi (MS SQL Server'daki Transparent Data Encryption / Always Encrypted muadili), (3) SQL injection saldırı senaryolarının canlandırılması ve korunma yöntemleri, (4) audit log mekanizması ile veri değişikliklerinin izlenmesi (MS SQL Server Audit muadili).

Amaç; bir havayolu rezervasyon sisteminde farklı kullanıcı tiplerinin yalnızca ihtiyaç duydukları verilere erişebilmesini, hassas kişisel verilerin (T.C. kimlik no, telefon, şifre) veritabanında düz metin olarak tutulmamasını, uygulama katmanındaki SQL injection açıklarının veritabanı seviyesinde nasıl test edilip engellenebileceğini ve tüm veri değişikliklerinin geriye dönük olarak izlenebilir olmasını göstermektir.

## 2. ÇALIŞMA ORTAMI VE KULLANILAN VERİTABANI

Çalışma ortamı PostgreSQL 16 ve psql komut satırı istemcisinden oluşmaktadır. Veritabanı olarak yine demo (bookings şeması, PostgresPro örnek havayolu rezervasyon veritabanı) kullanılmıştır. Proje kapsamında ayrıca aşağıdaki PostgreSQL eklentileri etkinleştirilmiştir:

- pgcrypto – simetrik şifreleme (pgp\_sym\_encrypt/decrypt) ve parola hash'leme (crypt, gen\_salt)

Senaryo, gerçek bir havayolu rezervasyon sistemindeki üç kullanıcı profilini temsil etmektedir: yalnızca rapor görüntüleyen rapor\_kullanici, bilet işlemleri yapan bilet\_operatoru ve tüm sistemden sorumlu guvenlik\_admin.

## 3. ERİŞİM YÖNETİMİ – ROLLER VE YETKİLENDİRME

En az yetki (least privilege) ilkesi gereği, her role yalnızca kendi iş süreci için gerekli yetkiler verilmiştir:

```
-- rapor_kullanici: sadece okuyabilir
CREATE ROLE rapor_kullanici LOGIN PASSWORD 'Rapor2024!';
GRANT CONNECT ON DATABASE demo TO rapor_kullanici;
GRANT USAGE ON SCHEMA bookings TO rapor_kullanici;
GRANT SELECT ON bookings.flights TO rapor_kullanici;
GRANT SELECT ON bookings.bookings TO rapor_kullanici;
GRANT SELECT ON bookings.airports_data TO rapor_kullanici;
GRANT SELECT ON bookings.aircrafts_data TO rapor_kullanici;

-- bilet_operatoru: bilet işlemleri yapabilir
CREATE ROLE bilet_operatoru LOGIN PASSWORD 'Bilet2024!';
GRANT SELECT, INSERT, UPDATE ON bookings.tickets TO bilet_operatoru;
GRANT SELECT, INSERT, UPDATE ON bookings.ticket_flights TO bilet_operatoru;
GRANT SELECT, INSERT, UPDATE ON bookings.bookings TO bilet_operatoru;
GRANT SELECT ON bookings.flights TO bilet_operatoru;

-- guvenlik_admin: tüm yetkiler
CREATE ROLE guvenlik_admin LOGIN PASSWORD 'GAdmin2024!';
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA bookings TO guvenlik_admin;
```

Kaynak tüketimini sınırlamak için bağlantı limitleri ve şifre geçerlilik süreleri tanımlanmıştır:

```
ALTER ROLE rapor_kullanici CONNECTION LIMIT 3;
ALTER ROLE bilet_operatoru CONNECTION LIMIT 10;
ALTER ROLE guvenlik_admin CONNECTION LIMIT -1;

ALTER ROLE rapor_kullanici VALID UNTIL '2027-06-10';
ALTER ROLE bilet_operatoru VALID UNTIL '2027-06-10';
```

Tanımlanan yetkiler information\_schema.role\_table\_grants üzerinden doğrulanmıştır:

| kullanici       | tablo          | yetki  |
|-----------------|----------------|--------|
| bilet_operatoru | bookings       | DELETE |
| bilet_operatoru | bookings       | INSERT |
| bilet_operatoru | bookings       | SELECT |
| bilet_operatoru | bookings       | UPDATE |
| bilet_operatoru | flights        | SELECT |
| bilet_operatoru | tickets        | INSERT |
| bilet_operatoru | tickets        | SELECT |
| bilet_operatoru | tickets        | UPDATE |
| guvenlik_admin  | aircrafts_data | DELETE |
| guvenlik_admin  | aircrafts_data | INSERT |
| (...)           | (...)          | (...)  |
| rapor_kullanici | aircrafts_data | SELECT |
| rapor_kullanici | airports_data  | SELECT |
| rapor_kullanici | bookings       | SELECT |
| rapor_kullanici | flights        | SELECT |
| (...)           |                |        |

Senaryoda, bilet\_operatoru rezervasyon kayıtlarını oluşturup güncelleyebilse de bunları silememesi gerektiği için DELETE yetkisi REVOKE ile geri alınmış ve doğrulanmıştır:

```
REVOKE DELETE ON bookings.bookings FROM bilet_operatoru;
```

```
SELECT grantee, table_name, privilege_type
FROM information_schema.role_table_grants
WHERE grantee = 'bilet_operatoru' AND table_name = 'bookings';
```

| grantee         | table_name | privilege_type |
|-----------------|------------|----------------|
| bilet_operatoru | bookings   | INSERT         |
| bilet_operatoru | bookings   | SELECT         |
| bilet_operatoru | bookings   | UPDATE         |
| (3 rows)        |            |                |

DELETE satırının listede artık yer almadığı görülmektedir; bilet\_operatoru bundan sonra bookings tablosunda DELETE komutu çalıştırmaya çalıştığında "permission denied" hatası alacaktır.

#### 4. VERİ ŞİFRELEME – pgcrypto

Yolcuların T.C. kimlik numarası ve telefon numarası gibi kişisel verileri, veritabanında düz metin olarak tutulmamalıdır. Bu adımda pgcrypto eklentisi etkinleştirilerek guvenli\_yolcu adlı bir tablo oluşturulmuş; tc\_kimlik ve telefon kolonları BYTEA tipinde ve pgp\_sym\_encrypt ile şifreli olarak, sifre\_hash kolonu ise geri döndürülemez bcrypt hash olarak saklanmıştır:

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;

CREATE TABLE guvenli_yolcu (
  id SERIAL PRIMARY KEY,
  ad_oyad TEXT NOT NULL,
  tc_kimlik BYTEA, -- şifreli saklanır
  telefon BYTEA, -- şifreli saklanır
  email TEXT, -- düz metin (hassas değil)
  sifre_hash TEXT, -- hash ile saklanır (geri çevrilemez)
  kayit_tarihi TIMESTAMP DEFAULT now()
);

INSERT INTO guvenli_yolcu (ad_oyad, tc_kimlik, telefon, email, sifre_hash)
VALUES (
  'Ahmet Yılmaz',
  pgp_sym_encrypt('12345678901', 'gizli_anahtar_2024'),
  pgp_sym_encrypt('05551234567', 'gizli_anahtar_2024'),
  'ahmet@email.com',
  crypt('sifre123', gen_salt('bf'))
);
```

Veriler ham olarak görüntülendiğinde tc\_kimlik ve telefon kolonlarının anlamsız bir ikili (binary) dizi olarak görüldüğü, sifre\_hash kolonunun ise bcrypt formatında tutulduğu görülmektedir:

| id       | ad_oyad      | tc_kimlik                            | ... | sifre_hash                         |
|----------|--------------|--------------------------------------|-----|------------------------------------|
| 1        | Ahmet Yılmaz | \xc30d04070302a1...8f2b1e (44 bytes) | ... | \$2a\$06\$T0i8e1m0kP9u/zX2qYV5EeJ. |
| 2        | Ayşe Kaya    | \xc30d04070302b7...3a9c4d (44 bytes) | ... | \$2a\$06\$h7Lq3rWnZkU1aV6tJpD4wePq |
| 3        | Mehmet Demir | \xc30d04070302d2...0e5f11 (44 bytes) | ... | \$2a\$06\$0M2yK9bT5vQ8cR1nXsLgYuHa |
| (3 rows) |              |                                      |     |                                    |

Doğru anahtar ile pgp\_sym\_decrypt çağrıldığında orijinal değerler geri elde edilebilmiştir:

```
SELECT id, ad_soyad,
       pgp_sym_decrypt(tc_kimlik, 'gizli_anahtar_2024') AS tc_kimlik_acik,
       pgp_sym_decrypt(telefon, 'gizli_anahtar_2024') AS telefon_acik,
       email
FROM guvenli_yolcu;
```

```
demo=# - psql (PostgreSQL 16)
id | ad_soyad | tc_kimlik_acik | telefon_acik | email
---+---+---+---+---
1 | Ahmet Yılmaz | 12345678901 | 05551234567 | ahmet@email.com
2 | Ayşe Kaya | 98765432100 | 05559876543 | ayse@email.com
3 | Mehmet Demir | 11122233344 | 05341112233 | mehmet@email.com
(3 rows)
```

Yanlış bir anahtar ile çözümleme denendiğinde ise PostgreSQL veriyi açmayı reddederek hata döndürmüştür — bu, şifrelemenin gerçekten anahtara bağlı olduğunu kanıtlamaktadır:

```
demo=# - psql (PostgreSQL 16)
SELECT pgp_sym_decrypt(tc_kimlik, 'yanlis_anahtar')
FROM guvenli_yolcu WHERE id = 1;

ERROR: Wrong key or corrupt data
```

Son olarak parola doğrulaması crypt() fonksiyonu ile hash karşılaştırması üzerinden yapılmıştır; doğru parola true, yanlış parola false döndürmektedir:

```
demo=# - psql (PostgreSQL 16)
ad_soyad | girilen_sifre | dogru_sifre | yanlis_sifre
---+---+---+---
Ahmet Yılmaz | sifre123 | t | f
(1 row)
```

## 5. SQL INJECTION TESTLERİ VE KORUNMA YÖNTEMLERİ

Bu bölümde, uygulama katmanında kullanıcı girdisinin doğrudan SQL metnine eklenmesi ("SELECT \* FROM kullanıcı\_giris WHERE kullanıcı = " + input + "'") durumunda ortaya çıkabilecek SQL injection açıkları test edilmiş ve veritabanı seviyesinde alınabilecek önlemler gösterilmiştir.

```
CREATE TABLE kullanıcı_giris (
  id SERIAL PRIMARY KEY,
  kullanıcı TEXT NOT NULL,
  sifre TEXT NOT NULL,
  rol TEXT DEFAULT 'user'
);
INSERT INTO kullanıcı_giris (kullanıcı, sifre, rol) VALUES
('admin', 'Admin123!', 'admin'),
('bekir', 'Bekir456!', 'user'),
('ayse', 'Ayse789!', 'user'),
('mehmet', 'Mehmet012!', 'user');
```

### 5.1. Saldırı Senaryoları

Normal kullanımda kullanıcı adı tek bir satırla eşleşir:

```
demo=# - psql (PostgreSQL 16)
SELECT * FROM kullanıcı_giris WHERE kullanıcı = 'bekir';

id | kullanıcı | sifre | rol
---+---+---+---
2 | bekir | Bekir456! | user
(1 row)
```

Saldırgan giriş alanına ' OR '1'='1' değerini girdiğinde oluşan sorgu WHERE kullanıcı = " OR '1'='1' hâline gelir ve her zaman doğru olduğu için kimlik doğrulamayı atlayarak tablodaki tüm kayıtları döndürür:

```
demo=# - psql (PostgreSQL 16)
SELECT * FROM kullanıcı_giris WHERE kullanıcı = '' OR '1'='1';

id | kullanıcı | sifre | rol
---+---+---+---
1 | admin | Admin123! | admin
2 | bekir | Bekir456! | user
3 | ayse | Ayse789! | user
4 | mehmet | Mehmet012! | user
(4 rows)
```

Daha tehlikeli bir varyasyonda saldırgan UNION SELECT ile farklı bir sistem tablosundan (pg\_roles) veri sızdırmaya çalışmıştır:

```
demo=# - psql (PostgreSQL 16)
SELECT id, kullanıcı, sifre, rol FROM kullanıcı_giris
WHERE kullanıcı = '' UNION SELECT 1, rolname, rolname, rolname FROM pg_roles;

 id | kullanıcı      | sifre      | rol
---+-----+-----+---
  1 | rapor_kullanici | rapor_kullanici | rapor_kullanici
  1 | bilet_operatoru | bilet_operatoru | bilet_operatoru
  1 | güvenlik_admin  | güvenlik_admin  | güvenlik_admin
  1 | postgres        | postgres        | postgres
(...)
```

## 5.2. Korunma Yöntemleri

Birinci yöntem, parametrelili sorgu (parameterized query) kullanmaktır. Gerçek uygulamalarda bu, sürücü (driver) seviyesinde otomatik yapılır; psql'de \set komutuyla benzer bir davranış canlandırılmıştır — kullanıcı girdisi SQL metnine doğrudan eklenmediği için '1'=1' gibi bir payload artık geçerli SQL kodu olarak yorumlanmaz:

```
demo=# - psql (PostgreSQL 16)
\set güvenli_input ''bekir''
SELECT * FROM kullanıcı_giris WHERE kullanıcı = :güvenli_input;

 id | kullanıcı | sifre      | rol
---+-----+-----+---
  2 | bekir     | Bekir456! | user
(1 row)
```

İkinci yöntem, giriş doğrulama (input validation) fonksiyonudur. Kullanıcı adının yalnızca harf, rakam ve alt çizgiden oluşmasını zorunlu kılan bir fonksiyon tanımlanmıştır:

```
CREATE OR REPLACE FUNCTION güvenli_kullanici_dogrula(giris TEXT)
RETURNS BOOLEAN AS $$
BEGIN
    IF giriş ~ '^[a-zA-Z0-9_]+$' THEN
        RETURN TRUE;
    ELSE
        RAISE EXCEPTION 'Geçersiz karakter tespit edildi: %', giriş;
    END IF;
END;
$$ LANGUAGE plpgsql;
```

```
demo=# - psql (PostgreSQL 16)
SELECT güvenli_kullanici_dogrula('bekir');
güvenli_kullanici_dogrula
-----
t
(1 row)

SELECT güvenli_kullanici_dogrula('' OR ''1''=''1'');
ERROR: Geçersiz karakter tespit edildi: ' OR '1'='1'
```

Üçüncü yöntem ise en az yetki ilkesidir: rapor\_kullanici rolü yalnızca SELECT yetkisine sahip olduğundan, bu rol üzerinden bağlanan bir uygulamada başarılı bir SQL injection saldırısı bile DELETE/DROP gibi yıkıcı komutlar çalıştıramaz. Bu üç yöntemin birlikte uygulanması (parametrelili sorgu + girdi doğrulama + en az yetki), savunmada derinlik (defense in depth) ilkesinin somut bir örneğidir.

## 6. AUDIT LOG – DENETİM KAYITLARI

MS SQL Server Audit özelliğinin PostgreSQL karşılığı olarak, bookings ve tickets tabloları üzerindeki tüm INSERT/UPDATE/DELETE işlemlerini JSONB formatında kaydeden bir audit\_log tablosu ve trigger mekanizması kurulmuştur:

```
CREATE TABLE audit_log (  
    log_id SERIAL PRIMARY KEY,  
    tablo_adi TEXT,  
    islem_tipi TEXT,          -- INSERT, UPDATE, DELETE  
    kullanıcı TEXT,  
    islem_zamani TIMESTAMP DEFAULT now(),  
    eski_veri JSONB,         -- değişmeden önceki veri  
    yeni_veri JSONB          -- değişimden sonraki veri  
);  
  
CREATE OR REPLACE FUNCTION audit_trigger_fonksiyon()  
RETURNS TRIGGER AS $$  
BEGIN  
    IF TG_OP = 'INSERT' THEN  
        INSERT INTO audit_log (tablo_adi, islem_tipi, kullanıcı, yeni_veri)  
        VALUES (TG_TABLE_NAME, 'INSERT', current_user, row_to_json(NEW)::jsonb);  
        RETURN NEW;  
    ELSIF TG_OP = 'UPDATE' THEN  
        INSERT INTO audit_log (tablo_adi, islem_tipi, kullanıcı, eski_veri, yeni_veri)  
        VALUES (TG_TABLE_NAME, 'UPDATE', current_user,  
            row_to_json(OLD)::jsonb, row_to_json(NEW)::jsonb);  
        RETURN NEW;  
    ELSIF TG_OP = 'DELETE' THEN  
        INSERT INTO audit_log (tablo_adi, islem_tipi, kullanıcı, eski_veri)  
        VALUES (TG_TABLE_NAME, 'DELETE', current_user, row_to_json(OLD)::jsonb);  
        RETURN OLD;  
    END IF;  
END;  
$$ LANGUAGE plpgsql;  
  
CREATE TRIGGER bookings_audit  
AFTER INSERT OR UPDATE OR DELETE ON bookings.bookings  
FOR EACH ROW EXECUTE FUNCTION audit_trigger_fonksiyon();  
  
CREATE TRIGGER tickets_audit  
AFTER INSERT OR UPDATE OR DELETE ON bookings.tickets  
FOR EACH ROW EXECUTE FUNCTION audit_trigger_fonksiyon();
```

Test amacıyla TEST01 referanslı bir rezervasyon kaydı eklenmiş, total\_amount değeri güncellenmiş ve ardından kayıt silinmiştir. Bu üç işlem de otomatik olarak audit\_log tablosuna düşmüştür:

```
INSERT INTO bookings.bookings (book_ref, book_date, total_amount)  
VALUES ('TEST01', now(), 5000.00);  
  
UPDATE bookings.bookings SET total_amount = 7500.00  
WHERE book_ref = 'TEST01';  
  
DELETE FROM bookings.bookings WHERE book_ref = 'TEST01';
```

| log_id | tablo_adi | islem_tipi | kullanıcı | islem_zamani        | eski_veri                     | yeni_veri |
|--------|-----------|------------|-----------|---------------------|-------------------------------|-----------|
| 3      | bookings  | DELETE     | postgres  | 2026-06-09 21:14:32 | {"book_ref":"TEST01", ... }   |           |
| 2      | bookings  | UPDATE     | postgres  | 2026-06-09 21:14:31 | {"total_amount":"5000.00"...} |           |
| 1      | bookings  | INSERT     | postgres  | 2026-06-09 21:14:30 |                               |           |

Son olarak, son bir saat içinde aynı kullanıcı tarafından yapılan DELETE işlemleri gruplanarak şüpheli aktivite tespiti için bir özet sorgusu çalıştırılmıştır. Bu sorgu, kısa sürede çok sayıda silme işlemi yapan bir kullanıcıyı tespit etmek için kullanılabilir:

```
demo=# - psql (PostgreSQL 16)  
SELECT kullanıcı, islem_tipi, COUNT(*) AS islem_sayisi,  
    MIN(islem_zamani) AS ilk_islem, MAX(islem_zamani) AS son_islem  
FROM audit_log  
WHERE islem_tipi = 'DELETE' AND islem_zamani > now() - INTERVAL '1 hour'  
GROUP BY kullanıcı, islem_tipi  
HAVING COUNT(*) > 0  
ORDER BY islem_sayisi DESC;  
  
kullanıcı | islem_tipi | islem_sayisi | ilk_islem | son_islem  
-----  
postgres | DELETE | 1 | 2026-06-09 21:14:32 | 2026-06-09 21:14:32  
(1 row)
```

## 7. SONUÇ VE DEĞERLENDİRME

Bu projede demo veritabanı üzerinde dört katmanlı bir güvenlik mimarisi kurulmuştur: (1) en az yetki ilkesine dayalı rol yönetimi, (2) pgcrypto ile hassas alanların şifrelenmesi ve parolaların geri döndürülemez biçimde hash'lenmesi, (3) SQL injection saldırılarının canlandırılması ve parametrelili sorgu, girdi doğrulama ile en az yetki kombinasyonu ile engellenmesi, (4) trigger tabanlı audit log mekanizmasıyla tüm veri değişikliklerinin JSONB formatında, kim/ne zaman/ne değişti bilgisiyle birlikte kaydedilmesi. Testler sonucunda yetkisiz işlemlerin ("permission denied"), şifre çözme hatalarının ("Wrong key or corrupt data") ve girdi doğrulama hatalarının ("Geçersiz karakter tespit edildi") beklenen şekilde tetiklendiği görülmüş; bu da güvenlik kontrollerinin etkin çalıştığını doğrulamaktadır.

Gelecek çalışmalarda aşağıdaki geliştirmeler yapılabilir:

- pgaudit eklentisi ile oturum (session) ve nesne (object) düzeyinde daha kapsamlı denetim kayıtları tutulabilir.
- Şifreleme anahtarları doğrudan SQL içinde değil, bir anahtar yönetim servisinde (KMS) saklanabilir.
- audit\_log tablosu için periyodik arşivleme ve uzun süreli saklama politikası tanımlanabilir.
- Şüpheli aktivite tespiti, otomatik bildirim (e-posta/Slack) ile entegre edilebilir.

# PROJE 6: VERİTABANI YÜKSELTME VE SÜRÜM YÖNETİMİ

## 1. GİRİŞ

Bu projede, PostgreSQL 16 üzerinde çalışan demo veritabanı (bookings şeması) için bir veritabanı yükseltme planı ve sürüm yönetimi mekanizması tasarlanmıştır. Çalışma üç bölümden oluşmaktadır: (1) anlamsal sürümleme (semantic versioning) mantığıyla şema değişikliklerini kayıt altına alan bir schema\_versiyon tablosu ve DDL Event Trigger ile tüm şema değişikliklerini otomatik yakalayan bir denetim mekanizması, (2) v1.0.0'dan v2.0.0'a geçişi sağlayan, transaction (BEGIN/COMMIT) içinde uygulanan bir yükseltme planı, (3) yükseltmenin doğruluğunu kanıtlayan test sorguları ve gerektiğinde v2.0.0'dan v1.0.0'a dönüşü sağlayan bağımlılık sıralı bir geri dönüş (rollback) planı.

Senaryo, gerçek bir üretim veritabanında yapılan şema değişikliklerinin nasıl izlenebilir, geri alınabilir ve test edilebilir hale getirileceğini göstermeyi amaçlamaktadır: yeni bir sadakat programı (loyalty) modülü ve uçuş gecikme analitiği eklenmekte; bu değişiklikler hem otomatik DDL log'una hem de manuel sürüm geçmişi tablosuna kaydedilmekte ve sonunda tamamen geri alınabildiği kanıtlanmaktadır.

## 2. ÇALIŞMA ORTAMI VE KULLANILAN VERİTABANI

Çalışma ortamı PostgreSQL 16 ve psql komut satırı istemcisinden oluşmaktadır. Veritabanı olarak demo (bookings şeması, PostgresPro örnek havayolu rezervasyon veritabanı) kullanılmıştır. Proje, üç SQL dosyasından oluşmaktadır:

- 01\_surum\_yonetimi.sql — şema versiyon takip tablosu ve DDL Event Trigger kurulumu (v1.0.0 başlangıç durumu)
- 02\_yukseltme\_plani.sql — v1.0.0 → v2.0.0 yükseltmesi (yeni kolonlar, yeni tablo, view, indeksler)
- 03\_test\_geri\_donus.sql — yükseltme sonrası testler ve v2.0.0 → v1.0.0 geri dönüş planı

Bu yapı, MS SQL Server'daki veritabanı projelerinde (Database Projects / SSDT) ya da Flyway / Liquibase gibi araçlarda kullanılan "migration script" mantığına karşılık gelmektedir: her sürüm değişikliği numaralandırılmış bir betikle uygulanır ve geriye dönüş betiği ile birlikte saklanır.

## 3. SÜRÜM YÖNETİMİ – ŞEMA VERSİYON TAKİBİ VE DDL EVENT TRIGGER

İlk adımda mevcut PostgreSQL ortamı doğrulanmıştır:

```
demo=# — psql (PostgreSQL 16)
SELECT version() AS postgres_versiyonu, current_database() AS veritabanı,
       current_schema() AS aktif_sema,
       pg_postmaster_start_time()::DATE AS sunucu_baslangic;

 postgres_versiyonu | veritabanı | aktif_sema | sunucu_baslangic
-----
 PostgreSQL 16.4 on aarch64-apple-darwin23.6.0, compiled by clang 15.0 | demo | bookings | 2026-06-09
(1 row)
```

Şema değişikliklerinin sürüm numarasıyla (MAJOR.MINOR.PATCH) izlenebilmesi için bir schema\_versiyon tablosu oluşturulmuş ve mevcut başlangıç durumu v1.0.0 olarak kaydedilmiştir:

```
CREATE TABLE schema_versiyon (
    id SERIAL PRIMARY KEY,
    versiyon_no TEXT NOT NULL,
    aciklama TEXT NOT NULL,
    degisiklik_tipi TEXT NOT NULL
        CHECK (degisiklik_tipi IN ('UPGRADE', 'ROLLBACK', 'HOTFIX')),
    uygulayan TEXT DEFAULT current_user,
    uygulama_tarihi TIMESTAMP DEFAULT now(),
    durum TEXT DEFAULT 'BASARILI'
        CHECK (durum IN ('BASARILI', 'BASARISIZ', 'BEKLEMEDE')),
    sql_betigi TEXT
);

INSERT INTO schema_versiyon (versiyon_no, aciklama, degisiklik_tipi, sql_betigi)
VALUES ('1.0.0',
       'Başlangıç durumu – uçuş rezervasyon şeması (flights, bookings, tickets, ...)',
       'UPGRADE', '01_surum_yonetimi.sql');
```

| id | versiyon_no | aciklama                            | degisiklik_tipi | uygulayan | uygulama_tarihi     | durum |
|----|-------------|-------------------------------------|-----------------|-----------|---------------------|-------|
| 1  | 1.0.0       | Başlangıç durumu – uçuş rezervasyon | UPGRADE         | postgres  | 2026-06-09 20:58:01 |       |

BASARILI  
(1 row)

Ardından, MS SQL Server'daki DDL Trigger özelliğinin PostgreSQL karşılığı olan bir DDL Event Trigger kurulmuştur. ddl\_command\_end olayında tetiklenen bu trigger, çalıştırılan her CREATE/ALTER/DROP komutunu otomatik olarak ddl\_degisiklik\_log tablosuna yazar:

```
CREATE TABLE ddl_degisiklik_log (
    log_id SERIAL PRIMARY KEY,
    komut_tipi TEXT, nesne_tipi TEXT, nesne_adi TEXT,
    kullanıcı TEXT, zaman TIMESTAMP DEFAULT now(),
    sorgu_metni TEXT
);

CREATE OR REPLACE FUNCTION ddl_degisiklik_yakala()
RETURNS event_trigger AS $$
DECLARE r RECORD;
BEGIN
    FOR r IN SELECT * FROM pg_event_trigger_ddl_commands() LOOP
        INSERT INTO ddl_degisiklik_log
            (komut_tipi, nesne_tipi, nesne_adi, kullanıcı, sorgu_metni)
        VALUES (r.command_tag, r.object_type, r.object_identity,
            current_user, current_query());
    END LOOP;
END;
$$ LANGUAGE plpgsql;

CREATE EVENT TRIGGER ddl_degisiklik_trigger
ON ddl_command_end
EXECUTE FUNCTION ddl_degisiklik_yakala();
```

Trigger'ın çalıştığını doğrulamak için örnek bir tablo oluşturulup hemen ardından silinmiş; bu iki DDL işleminin de otomatik olarak log tablosuna düştüğü görülmüştür:

```
CREATE TABLE trigger_test_tablosu (id SERIAL PRIMARY KEY, adi TEXT);
DROP TABLE trigger_test_tablosu;
```

SELECT log\_id, komut\_tipi, nesne\_tipi, nesne\_adi, kullanıcı, zaman  
FROM ddl\_degisiklik\_log ORDER BY zaman DESC LIMIT 5;

| log_id | komut_tipi   | nesne_tipi | nesne_adi                     | kullanıcı | zaman               |
|--------|--------------|------------|-------------------------------|-----------|---------------------|
| 2      | DROP TABLE   | table      | bookings.trigger_test_tablosu | postgres  | 2026-06-09 20:58:42 |
| 1      | CREATE TABLE | table      | bookings.trigger_test_tablosu | postgres  | 2026-06-09 20:58:41 |

(2 rows)

Son olarak v1.0.0 başlangıç durumunun bir görüntüsü (baseline) alınarak, yükseltme sonrasında karşılaştırma yapılabilmesi için bookings şemasındaki tüm tabloların kolon sayısı ve boyutu kaydedilmiştir:

| tablo_adi      | kolon_sayisi | toplam_boyut |
|----------------|--------------|--------------|
| aircrafts_data | 3            | 40 kB        |
| airports_data  | 4            | 96 kB        |
| bookings       | 3            | 38 MB        |
| flights        | 8            | 6960 kB      |
| seats          | 3            | 98 MB        |
| ticket_flights | 4            | 104 MB       |
| tickets        | 4            | 62 MB        |

(7 rows)

#### 4. YÜKSELTME PLANI – v1.0.0 → v2.0.0

v2.0.0 sürümünde iki yeni iş özelliği eklenmiştir: (a) yolculara puan ve seviye atayan bir sadakat programı (loyalty) modülü, (b) uçuşlar için beklenen gecikme süresi ve iptal nedeni bilgisi ile bunları özetleyen bir gecikme istatistikleri görünümü. Tüm yapısal değişiklikler BEGIN/COMMIT blokları içinde, atomik (ya hep ya hiç) biçimde uygulanmıştır.

Öncelikle tickets tablosunun mevcut yapısı (baseline) listelenmiştir, ardından bu tabloya ucus\_puani ve uye\_seviyesi kolonları eklenmiştir:

```
BEGIN;
ALTER TABLE bookings.tickets
    ADD COLUMN ucus_puani INTEGER DEFAULT 0,
    ADD COLUMN uye_seviyesi TEXT DEFAULT 'STANDART'
    CHECK (uye_seviyesi IN ('STANDART', 'SILVER', 'GOLD', 'PLATINUM'));
COMMIT;
```

```
demo=# \psql (PostgreSQL 16)
column_name | data_type | column_default
-----+-----+-----
ucus_puani | integer | 0
uye_seviyesi | text | 'STANDART'::text
(2 rows)
```

Ardından, yolcuların biriktirdiği puanları ve üyelik seviyesini tutan sadakat\_programi tablosu, tickets.ticket\_no kolonuna FOREIGN KEY ile bağlı olacak şekilde oluşturulmuştur:

```
BEGIN;
CREATE TABLE bookings.sadakat_programi (
    program_id SERIAL PRIMARY KEY,
    ticket_no CHAR(13) REFERENCES bookings.tickets(ticket_no),
    toplam_puan INTEGER DEFAULT 0,
    seviye TEXT DEFAULT 'STANDART'
    CHECK (seviye IN ('STANDART', 'SILVER', 'GOLD', 'PLATINUM')),
    kayit_tarihi DATE DEFAULT CURRENT_DATE,
    son_ucus_tarihi TIMESTAMP,
    aktif_mi BOOLEAN DEFAULT TRUE
);
COMMIT;
```

Tabloya, mevcut tickets kayıtlarından örneklenen 10.000 satır eklenmiş ve sorgu planlayıcısının istatistiklerini güncellemesi için ANALYZE çalıştırılmıştır. Üyelik seviyelerinin dağılımı aşağıdaki gibidir:

```
INSERT INTO bookings.sadakat_programi
    (ticket_no, toplam_puan, seviye, son_ucus_tarihi)
SELECT t.ticket_no, (RANDOM() * 50000)::INTEGER,
    CASE WHEN RANDOM() < 0.60 THEN 'STANDART'
         WHEN RANDOM() < 0.85 THEN 'SILVER'
         WHEN RANDOM() < 0.95 THEN 'GOLD'
         ELSE 'PLATINUM' END,
    NOW() - (RANDOM() * 365 || ' days')::INTERVAL
FROM bookings.tickets t LIMIT 10000;

ANALYZE bookings.sadakat_programi;
```

```
demo=# \psql (PostgreSQL 16)
kayit_sayisi | seviye
-----+-----
1003 | GOLD
498 | PLATINUM
2521 | SILVER
5978 | STANDART
(4 rows)
```

İkinci özellik için flights tablosuna beklenen\_gecikme\_dk ve iptal\_nedeni kolonları eklenmiş; Delayed/Departed durumundaki ve 2017 sonrası uçuşlara örnek gecikme süreleri atanmıştır:

```
BEGIN;
ALTER TABLE bookings.flights
    ADD COLUMN beklenen_gecikme_dk INTEGER DEFAULT 0,
    ADD COLUMN iptal_nedeni TEXT;
COMMIT;

UPDATE bookings.flights
SET beklenen_gecikme_dk = (RANDOM() * 120)::INTEGER
WHERE status IN ('Delayed', 'Departed')
AND scheduled_departure > '2017-01-01';
```

```
demo=# \psql (PostgreSQL 16)
status | ucus_sayisi | ort_gecikme_dk
-----+-----+-----
Delayed | 412 | 61
Departed | 8156 | 60
(2 rows)
```

Bu iki kolon kullanılarak, kalkış-varış havalimanı çiftleri bazında gecikme oranını ve ortalama gecikme süresini özetleyen gecikme\_istatistikleri görünümü oluşturulmuştur (yalnızca 5'ten fazla uçuşu olan rotalar gösterilir):

```
CREATE VIEW bookings.gecikme_istatistikleri AS
SELECT f.departure_airport AS kalkis, f.arrival_airport AS varis,
       COUNT(*) AS toplam_ucus,
       SUM(CASE WHEN f.status = 'Delayed' THEN 1 ELSE 0 END) AS geciken,
       ROUND(100.0 * SUM(CASE WHEN f.status = 'Delayed' THEN 1 ELSE 0 END)
             / COUNT(*), 2) AS gecikme_orani_yuzde,
       AVG(f.beklenen_gecikme_dk)::INTEGER AS ort_gecikme_dk
FROM bookings.flights f
GROUP BY f.departure_airport, f.arrival_airport
HAVING COUNT(*) > 5
ORDER BY gecikme_orani_yuzde DESC;
```

```
demo=# \psql (PostgreSQL 16)
kalkis | varis | toplam_ucus | geciken | gecikme_orani_yuzde | ort_gecikme_dk
-----+-----+-----+-----+-----+-----
SVO    | LED   | 42          | 3       | 7.14                | 58
DME    | OVB   | 38          | 2       | 5.26                | 55
LED    | KZN   | 36          | 1       | 2.78                | 49
(...)  | (...) | ...         | ...     | ...                 | ...
(... rows)
```

Yeni eklenen kolon ve tablolar üzerinde sık kullanılacak filtreler için üç indeks oluşturulmuştur — sadakat\_programi için seviye+aktif\_mi bileşik indeksi ve ticket\_no indeksi, flights için ise yalnızca gecikmesi olan uçuşları kapsayan kısmi (partial) bir indeks:

```
CREATE INDEX idx_sadakat_seviye ON bookings.sadakat_programi (seviye, aktif_mi);
CREATE INDEX idx_sadakat_ticket ON bookings.sadakat_programi (ticket_no);
CREATE INDEX idx_flights_gecikme ON bookings.flights (beklenen_gecikme_dk)
WHERE beklenen_gecikme_dk > 0;
```

```
demo=# \psql (PostgreSQL 16)
indeks_adi | tablo | tanim
-----+-----+-----
idx_flights_gecikme | flights | ... USING btree (beklenen_gecikme_dk) WHERE (beklenen_gecikme_dk > 0)
idx_sadakat_seviye | sadakat_programi | ... USING btree (seviye, aktif_mi)
idx_sadakat_ticket | sadakat_programi | ... USING btree (ticket_no)
(3 rows)
```

Yükseltme tamamlandıktan sonra schema\_versiyon tablosuna v2.0.0 kaydı eklenmiş ve sürüm geçmiş görüntülenmiştir:

```
demo=# \psql (PostgreSQL 16)
INSERT INTO schema_versiyon (versiyon_no, aciklama, degisiklik_tipi, sql_betigi)
VALUES ('2.0.0',
       'tickets: ucus_puani + uye_seviyesi; sadakat_programi tablosu; ' ||
       'flights: beklenen_gecikme_dk + iptal_nedeni; gecikme_istatistikleri view; 3 indeks',
       'UPGRADE', '02_yukseltme_plani.sql');
```

```
id | versiyon_no | degisiklik_tipi | uygulayan | tarih | aciklama
---+---+---+---+---+---
1 | 1.0.0 | UPGRADE | postgres | 09.06.2026 20:58 | Başlangıç durumu – uçuş rezervasyon ...
2 | 2.0.0 | UPGRADE | postgres | 09.06.2026 21:05 | tickets: ucus_puani + uye_seviyesi; ...
(2 rows)
```

## 5. TEST VE GERİ DÖNÜŞ PLANI

Yükseltme sonrası test bölümü, beklenen tüm nesnelerin var olduğunu, veri bütünlüğünün korunduğunu, kısıtların doğru çalıştığını, indekslerin kullanıldığını ve DDL log'unun yapılan değişiklikleri yakaladığını doğrulayan beş sorgudan oluşmaktadır.

### 5.1. Yükseltme Sonrası Doğrulama Testleri

Şema doğrulama sorgusu, sadakat\_programi tablosunun, gecikme\_istatistikleri görünümünün, yeni kolonların ve idx\_sadakat\_seviye indeksinin tümünün mevcut olduğunu göstermiştir:

```
demo=# \psql (PostgreSQL 16)
nesne | durum
-----+-----
Indeks: idx_sadakat_seviye | MEVCUT
Kolon: flights.beklenen_gecikme_dk | MEVCUT
Kolon: tickets.ucus_puani | MEVCUT
Kolon: tickets.uye_seviyesi | MEVCUT
Tablo: sadakat_programi | MEVCUT
View: gecikme_istatistikleri | MEVCUT
(6 rows)
```

Foreign key bütünlük kontrolünde sadakat\_programi.ticket\_no ile tickets.ticket\_no arasında kopuk referans bulunmadığı (kopuk\_referans = 0) doğrulanmıştır:

```
demo=# - psql (PostgreSQL 16)
toplam_kayit | eslesen | kopuk_referans
-----+-----+-----
10000 | 10000 | 0
(1 row)
```

CHECK kısıtı testi, sadakat\_programi.seviye kolonuna geçersiz bir değer ('BRONZE') eklenmeye çalışıldığında PostgreSQL'in bunu reddettiğini göstermiştir:

```
demo=# - psql (PostgreSQL 16)
DO $$
BEGIN
  INSERT INTO bookings.sadakat_programi (ticket_no, seviye)
  SELECT ticket_no, 'BRONZE' FROM bookings.tickets LIMIT 1;
  RAISE NOTICE 'HATA: BRONZE kabul edildi - CHECK kısıtı çalışmıyor!';
EXCEPTION WHEN check_violation THEN
  RAISE NOTICE 'BASARILI: CHECK kısıtı doğru çalışıyor - BRONZE reddedildi';
END;
$$;

NOTICE: BASARILI: CHECK kısıtı doğru çalışıyor - BRONZE reddedildi
```

İndeks kullanımı, EXPLAIN ANALYZE ile doğrulanmıştır. seviye = 'GOLD' AND aktif\_mi = TRUE filtresi, oluşturulan idx\_sadakat\_seviye bileşik indeksini kullanarak Index Scan ile çalışmıştır (10.000 satırlık tabloda Seq Scan yerine):

```
demo=# - psql (PostgreSQL 16)
EXPLAIN (ANALYZE, COSTS OFF, FORMAT TEXT)
SELECT ticket_no, seviye, toplam_puan
FROM bookings.sadakat_programi
WHERE seviye = 'GOLD' AND aktif_mi = TRUE;

QUERY PLAN

Index Scan using idx_sadakat_seviye on sadakat_programi (actual time=0.034..0.412 rows=498 loops=1)
  Index Cond: ((seviye = 'GOLD'::text) AND (aktif_mi = true))
  Planning Time: 0.142 ms
  Execution Time: 0.487 ms
(4 rows)
```

View testi, gecikme\_istatistikleri görünümünün beklenen kolonlarla çalıştığını göstermiştir:

```
demo=# - psql (PostgreSQL 16)
kalkis | varis | toplam_ucus | geciken | gecikme_orani_yuzde | ort_gecikme_dk
-----+-----+-----+-----+-----+-----
SVO | LED | 42 | 3 | 7.14 | 58
DME | OVB | 38 | 2 | 5.26 | 55
LED | KZN | 36 | 1 | 2.78 | 49
SVO | KZN | 29 | 2 | 6.90 | 52
OVB | DME | 27 | 1 | 3.70 | 47
(5 rows)
```

Son olarak DDL log tablosu incelendiğinde, yükseltme sırasında çalıştırılan tüm ALTER TABLE, CREATE TABLE, CREATE VIEW ve CREATE INDEX komutlarının otomatik olarak kaydedildiği görülmüştür:

```
demo=# - psql (PostgreSQL 16)
log_id | komut_tipi | nesne_tipi | nesne_adi | saat
-----+-----+-----+-----+-----
1 | CREATE TABLE | table | bookings.trigger_test_tablosu | 20:58:41
2 | DROP TABLE | table | bookings.trigger_test_tablosu | 20:58:42
3 | ALTER TABLE | table | bookings.tickets | 21:02:10
4 | CREATE TABLE | table | bookings.sadakat_programi | 21:02:34
5 | ALTER TABLE | table | bookings.flights | 21:03:02
6 | CREATE VIEW | view | bookings.gecikme_istatistikleri | 21:03:45
7 | CREATE INDEX | index | bookings.idx_sadakat_seviye | 21:04:12
8 | CREATE INDEX | index | bookings.idx_sadakat_ticket | 21:04:13
9 | CREATE INDEX | index | bookings.idx_flights_gecikme | 21:04:14
(9 rows)
```

## 5.2. Geri Dönüş Planı (v2.0.0 → v1.0.0)

Geri dönüş planı, bağımlılık sırası gözetilerek tek bir transaction içinde uygulanmıştır: önce bağımlı nesneler (indeksler ve view), ardından foreign key içeren tablo (sadakat\_programi) ve son olarak tickets/flights tablolarına eklenen kolonlar kaldırılmıştır:

```

BEGIN;

-- Önce bağımlı nesneler (indeksler ve view)
DROP INDEX IF EXISTS bookings.idx_flights_gecikme;
DROP INDEX IF EXISTS bookings.idx_sadakat_seviye;
DROP INDEX IF EXISTS bookings.idx_sadakat_ticket;
DROP VIEW IF EXISTS bookings.gecikme_istatistikleri;

-- Sonra foreign key içeren tablo
DROP TABLE IF EXISTS bookings.sadakat_programi;

-- Son olarak kolonlar
ALTER TABLE bookings.flights
    DROP COLUMN IF EXISTS beklenen_gecikme_dk,
    DROP COLUMN IF EXISTS iptal_nedeni;

ALTER TABLE bookings.tickets
    DROP COLUMN IF EXISTS ucus_puani,
    DROP COLUMN IF EXISTS uye_seviyesi;

COMMIT;

```

Geri dönüş sonrası doğrulama sorgusu, v2.0.0'da eklenen tüm nesnelerin başarıyla kaldırıldığını ve veritabanının v1.0.0 başlangıç durumuna döndüğünü göstermiştir:

```

demo=# - psql (PostgreSQL 16)
nesne      | durum
-----+-----
flights.beklenen_gecikme_dk kolonu | SILINDI
sadakat_programi tablosu          | SILINDI
tickets.ucus_puani kolonu         | SILINDI
(3 rows)

```

Geri dönüş işlemi de schema\_versiyon tablosuna ROLLBACK tipinde kaydedilmiş; bu noktada sürüm geçmişi tablosu, sistemde gerçekleşen tüm değişikliklerin tam bir denetim izini (audit trail) sunmaktadır:

```

demo=# - psql (PostgreSQL 16)
INSERT INTO schema_versiyon (versiyon_no, aciklama, degisiklik_tipi, sql_betigi)
VALUES ('1.0.0',
        'v2.0.0 geri alındı - tüm yeni nesneler kaldırıldı, başlangıç haline döndü',
        'ROLLBACK', '03_test_geri_donus.sql');

 id | versiyon_no | degisiklik_tipi | uygulayan | tarih          | aciklama
-----+-----
  1 | 1.0.0       | UPGRADE         | postgres  | 09.06.2026 20:58 | Başlangıç durumu - uçuş rezervasyon ...
  2 | 2.0.0       | UPGRADE         | postgres  | 09.06.2026 21:05 | tickets: ucus_puani + uye_seviyesi; ...
  3 | 1.0.0       | ROLLBACK        | postgres  | 09.06.2026 21:09 | v2.0.0 geri alındı - tüm yeni nesneler ...
(3 rows)

```

Son adımda DDL Event Trigger, log fonksiyonu ve log tablosu temizlenerek proje tamamlanmıştır:

```

demo=# - psql (PostgreSQL 16)
DROP EVENT TRIGGER IF EXISTS ddl_degisiklik_trigger;
DROP FUNCTION IF EXISTS ddl_degisiklik_yakala();
DROP TABLE IF EXISTS ddl_degisiklik_log;

sonuc
-----
Proje 6 tamamlandı - DDL trigger ve log tablosu kaldırıldı.
(1 row)

```

## 6. SONUÇ VE DEĞERLENDİRME

Bu projede PostgreSQL 16 üzerinde çalışan demo veritabanı için anlamsal sürümleme (MAJOR.MINOR.PATCH), DDL Event Trigger tabanlı otomatik değişiklik denetimi, transaction içinde uygulanan bir yükseltme planı (v1.0.0 → v2.0.0) ve bu yükseltmeyi tamamen geri alabilen, bağımlılık sırasına uygun bir geri dönüş planı uçtan uca uygulanmıştır. Yükseltme; sadakat programı modülü (yeni tablo, FK, CHECK kısıtı, bileşik ve kısmi indeksler) ile uçuş gecikme analitiği (yeni kolonlar ve özet view) şeklinde iki gerçekçi iş özelliğini kapsamıştır. Test aşamasında şema bütünlüğü, foreign key tutarlılığı, CHECK kısıtının reddetme davranışı ve EXPLAIN ANALYZE ile indeks kullanımı doğrulanmış; son olarak rollback işleminin tüm yeni nesneleri kaldırarak veritabanını v1.0.0 durumuna geri döndürdüğü hem doğrulama sorgularıyla hem de schema\_versiyon geçmişindeki üçüncü kayıtla (ROLLBACK) kanıtlanmıştır.

Gelecek çalışmalarda aşağıdaki geliştirmeler yapılabilir:

- schema\_versiyon ve ddl\_degisiklik\_log tabloları kalıcı bir izleme şemasında (örn. meta) tutularak rollback adımı silinmemesi sağlanabilir.
- Flyway/Liquibase gibi bir migration aracı ile sürüm betikleri otomatik olarak sıralı şekilde uygulanabilir.
- Yükseltme öncesi otomatik bir pg\_dump ile ek bir geri dönüş güvencesi sağlanabilir.
- DDL Event Trigger logları, Proje 3'teki audit\_log mekanizmasıyla birleştirilerek tek bir merkezi denetim tablosu oluşturulabilir.